

In the name of God
Machine Learning Course (Spring 2026)
Assignment 1

Due date: 19th of Farvardin

Studying Chapter 1 of Tom Michael's book is highly recommended before reading and implementing this assignment. In the first chapter of Tom Mitchel's book, the learning problem is reduced to a function approximation problem, and one specific class of learning Task is chosen to explain in detail.

Part I

In this assignment, you should use Mitchel's described approach to implement an algorithm to learn **the Snake game**. As you know, such tasks are abstract as a graph that every board state is a node, and every move is an edge.

The snake (player) attempts to eat items by running into them with the head of the snake. Each item eaten makes the snake longer, so avoiding collision with itself becomes progressively more difficult. The player loses when the snake runs into the screen border or itself or an obstacle.

The goal is to find the best movement for each state that helps the snake eats more items while consuming fewer movements.

We can define a function $V: S \rightarrow \mathbb{R}$ that assigns to every state a value and, based on V find the best move for each state. If we can find V , the problem is solved, but only a few pairs $\langle s, V(s) \rangle$ are available.

Many states do not have the value of V . Therefore, we should approximate this function only by these points (training examples). Because of the graph structure of the problem, this is possible to approximate the value of unknown-value states, and one can use approximated values to find better approximations. So for unknown-value states, you should use approximated successor state's value.

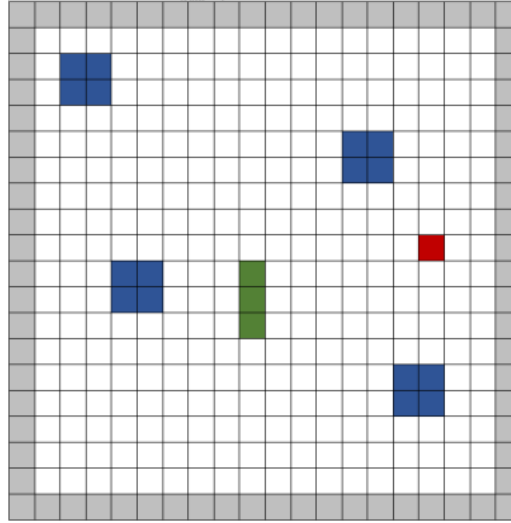
A- Implement an algorithm that learns to play Snake Game (20×20 board game) with a function approximation approach.

These steps will help you:

1. Choose linear representation for $\hat{V} : \hat{V}(\vec{x}) = \hat{\mathbf{w}}^T \vec{x}$.
2. Find some good features ($\vec{x}$) to describe the states of the Snake game.

In the name of God
Machine Learning Course (Spring 2026)
Assignment 1

3. Initialize all coefficients ($\hat{w}$) in $\hat{V}$ by zero.
4. Start with a random board state (a random place for the snake with a length of three (green cells) and four random places for four 2x2 obstacle blocks (blue cells) and a random place assigned to food (red cell)).



in each non-terminal state (S_t) choose your move based on $\hat{V}$ As follows: Among the moves the snake can do in the current state, choose the one which leads to the highest state value (**value of successors of the current state**).

Call these chosen successor state S_{t+1} and assign $V_{\text{train}}(S_t) = \hat{V}(S_{t+1})$. If S_t is a terminal state, assign $V_{\text{train}}(S_t) = \hat{V}(S)$

5. Set the value of the state as follows: (you may find other values by yourself)

$V_{\text{train}}(S_t) = -1000$ if the snake collisions with itself.

$V_{\text{train}}(S_t) = -1000$ if the snake collisions with an obstacle.

6. Now you have a new example: $\langle S_t, V_{\text{train}}(S_t) \rangle$. Based on this example update $\hat{w}$ as follows:

$$w_i = w_i + \alpha (V_{\text{train}}(S_t) - \hat{V}(S_t)) x_i$$

Assign a small value for α . Once you train the algorithm and find the weights, use these weights to play the game. (Save these codes in a separate file).

In the name of God
Machine Learning Course (Spring 2026)
Assignment 1

B- Optional Part:

1. The best agent (implementation) that can reach the most score is given extra scores.
2. Provide a graphical playing game for your implementation

Part II

A- Use all the described steps in previous part to implement an algorithm to learn Tic-tac-toe game.

Set the value state as follows:

$V_{train}(s_t) = 100$ If a player wins

$V_{train}(s_t) = -100$ If a player loses

$V_{train}(s_t) = 0$ If a player draws

B- Write an agent code that you can play with it. (Save this in a separate file).

Important Notes:

- Feel free to use your preferred programming languages.
- Pay extra attention to the due date. It will not be extended.
- Be advised that submissions after the deadline would not grade.
- Provide a report for your assignment and explain your features, code, and results.
- Make sure to upload only one zip file each group, the file must be named as: Student1_Student2.zip
- Any sign of cheating would result in a **zero** grade for this assignment.